

MetaData

The Fuse Plugin will also process and handle Metadata.

Metadata is a set of string variables that are attached to an image and are passed alongside pixel data through the comp. It is either loaded with an image from disk (the camera might have stored metadata inside the files), created or modified in your comp by certain tools and saved to disk in Saver tools (not every file format supports storing metadata though).

Viewing Metadata

To check what metadata is present in an image at a specific point inside your comp, you need to enable the metadata subview using the SubV button that is present below each image viewer. An overlay will list all the metadata (if any) that is present in the image in key/value pairs. Values can be numbers or strings but also tables when variables are nested.

A file's metadata is also displayed by the file requester when selecting a clip for a Loader tool.

Metadata and Fuses

This section describes what you need to know about metadata when writing Fuses.

The metadata is stored in a member variable of every Image object called `.Metadata` (it's nil if no metadata is available) that can both be read and written. Using the LUA functions `eyeon.writestring()` and `eyeon.readstring()`, you can place almost anything into the metadata that gets passed around.

To copy the metadata of your input image, it is recommended to use the `IMG_Like` attribute when creating your output image, even if you then define a different size or channel configuration. This will make sure the metadata of the input image is preserved.

All the metadata tools described above are Fuses so you can easily learn from their source code and adapt them for your own use cases. The `SparseColor` Fuse is another example of how to read and write metadata.

Supported File Types

Not every file type can store metadata. Traditionally, only cineon (.cin) and dpx files were used in conjunction with metadata and they can only store a predefined set of attributes. OpenEXR (.exr) files can be used to store any metadata you like and so can Fusion's .raw files that are used for disk caches. PNG files are able to store color space and gamut metadata. Other popular file formats like Tiff or Jpeg don't support metadata (at least Fusion can't save it to these file types).

List of known Metadata

This is a list of some of the metadata attributes that you might encounter, either because they come with certain image files or because they allow you to do nice things with them inside Fusion.

Metadata field name	Source	Description
Filename	Loader	This metadata is added by every Loader and contains the full path, frame number and extension of the currently loaded file.
FrameFilename	Loader	If the Loader points to an IFL file (text files, where each line contains the name of a file to be loaded), this variable will be added and contains the frame's actual filename while the Filename field will contain the (unchanging) path to the IFL file.
OriginalFilename	dpx files	Contains the "Filename" metadata of the source image that was subsequently saved as the dpx file you're currently viewing.
Project	dpx files	Contains the path and file name of the comp that rendered the current dpx file.
TimeCode	dpx files, mov exr	Contains the current frame's time code. For DPX this is usually generated by the camera and subsequently carried along through the pipeline unchanged. For Quicktime movies it depends on the mov's timecode track. You can use the SetTimeCode Fuse to set this manually. Format is HH:MM:SS:FF
FrameRate	mov	Contains the Quicktime movie's frame rate. SetTimeCode will also set this parameter but it may not be written to the output file.
CreationTime	dpx exr	Contains the real world's time when this frame was recorded by the camera.
Creator UserBits FramePosition Offset Center OriginalSize FilmFormat FrameID SlateInfo	Dpx exr	Various attributes you may encounter in dpx files
InputDevice InputSerial	dpx files and Saver tool	Fusion writes this to every dpx file.

Metadata field name	Source	Description
screenWindowWidth screenWindowCenter	exr files	These properties are part of the exr file standard and contain the position of the image's DoD
Gamma	dpx & png files	The image's gamma. Written to PNG files by the Saver tool but encountered in logarithmic dpx files as well where it might denote the conversion gamma (unconfirmed).
ColorSpace	Gamut tool	Contains the name of the output color space that has been selected in Fusion's Gamut tool.
WhitePointX WhitePointY RedChromaX RedChromaY GreenChromaX GreenChromaY BlueChromaX BlueChromaY	Gamut tool	These variables contain the coordinates of white point and color primaries as converted by the Gamut tool since the previously mentioned ColorSpace variable is just a user-friendly label (e.g. "sRGB") that contains little information for applications that deal with color science. These parameters can't be written to DPX files. Use OpenEXR instead. PNG files are also able to save this information.
Stereo.Method	Combiner tool	If Add Metadata is enabled in the Combiner tool, Fusion will attach this attribute to the output image to denote the stacking method of the current image. Valid values are "hstack" and "vstack" for horizontal and vertical stacking.
Stereo.Swap	manual	This property, if set to "true", will override the viewer's stereo display option. It's useful if you need to temporarily swap eyes somewhere inside your comp but you don't want to toggle the "swap eyes" option in the viewer by hand.
Stereo.Offset	manual	This "hidden" property needs to be set to an X and Y offset in pixels (e.g. {10, 0}) and will shift left and right eye when the viewer's stereo mode is enabled. It's very useful to add this property if your footage was shot with parallel cameras where zero disparity would occur on the horizon and everything would end up sticking out of the screen. Often, a preliminary per-shot depth grading value is supplied with the source footage to push the image back. Instead of actually transforming your left and right eye images and reverting this before the Saver, use Stereo.Offset like a viewer LUT: you'll be able to work on your raw footage while previewing the desired depth grade in the viewers.
nuke/node_hash	Nuke (exr files)	This is a checksum of the script that generated the image and is written to EXR files by Nuke.

Metadata Functions

The metadata is stored in a member variable for every Image called `.Metadata` (it's nil if no metadata is available) that can both be read and written. Using the functions `eyeon.writestring()` and `eyeon.readstring()` and the string functions of lua known and custom metadata can be processed with a fuse.

```
--[[--
This tool reads an image's metadata and prints to the console
--]]--

FuRegisterClass("ReadMetaData", CT_Tool, {
    REGS_Name = "Read Metadata",
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = "RMeta",
    REGS_OpDescription = "Reads Metadata and prints to console",
    REG_NoMotionBlurCtrls = true,
    REG_NoBlendCtrls = true,
    REG_OpNoMask = true,
    REG_NoPreCalcProcess = true,
    REG_SupportsDoD = true,
    REG_Fuse_NoJIT = true,
})

function Create()
    InImage = self:AddInput("Input", "Input", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

    OutImage = self:AddOutput("Output", "Output", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })
end

function Process(req)
    local img = InImage:GetValue(req)
    local result = img
    local meta

    for name, val in pairs(result.Metadata) do
        meta = result.Metadata[name]
        print(name, meta)
    end
    OutImage:Set(req, result)
end
```

Readstring

Summary

The `eyeon.readstring` will read a string from the metadata table and parse it into a table, separating strings and values into separate table entries

Usage

`eyeon.readstring(string val)`

val (string, required)

Example

This example updates metadata fields

```
function Process(req)
  local f_name = InFieldName:GetValue(req).Value
  local f_value = InFieldValue:GetValue(req).Value
  local img = InImage:GetValue(req)

  local result = Image({IMG_Like = img, IMG_NoData = req:IsPreCalc()})
  img:Crop(result, { })
  if (f_name ~= "") then
    local newmetadata = result.Metadata or {}
    if f_value ~= "" then
      -- create subtables for dotted field names
      local key
      local subtable = newmetadata
      for key in string.gmatch(f_name, "([%w_]+)%.") do
        subtable[key] = subtable[key] or {}
        subtable = subtable[key]
      end
      -- get final key
      f_name = string.match(f_name, "[%w_]+$")
      -- look for tables in f_value
      local tbl = string.match(f_value, "%s*%b{%}%s*")
      if tbl then
        subtable[f_name] = eyeon.readstring(f_value) --
        parse tables
      else
        subtable[f_name] = f_value
      end
    else
      newmetadata[f_name] = nil
    end
    result.Metadata = newmetadata
  end
  OutImage:Set(req, result)
end
```